CSE470: Software Engineering

Software Requirements Specification (SRS)

TutorNest

Prepared By

Student ID	Name
24301295	Rajesh Saha Roy
23201386	Aishwarya Roy
24301063	Aurpita Podder
24301399	Asir Bin Sayeed

Project Information

Section	11
Section Faculty	Nazmus Sakib Touhid(NAST)
GitHub Repository Link	https://github.com/aurps-codes/CSE470-Project
Tech Stack Used	FastAPI(Backend), REACT(Frontend), MongoDB(Database)

1. Introduction

1.1 Purpose

This document describes the requirements of TutorNest, a web-based platform that connects students with tutors. It notes down the system's features, users, and constraints so that the team and stakeholders share one proper understanding of what will be built and how it will be built.

1.2 Scope

TutorNest lets students and teachers post tuition requests online and connect each other meeting requirements. It supports one-to-one online classes through integrated video calls, with scheduling and payments handled on the platform. It also includes teacher verification, a student–teacher review system, and attendance and progress reports that parents can view. An AI assistant helps users navigate the website and answers frequently asked questions.

2. Overall Description

2.1 Product Perspective

TutorNest is a standalone web-based tuition marketplace that brings the whole tutoring process into one platform: finding a verified tutor, communicating, paying, scheduling classes, and giving feedback. It replaces the informal methods students and tutors use today, such as word of mouth and social media posts. It supports only online tuition, and it uses external services for payments, maps, calendar sync, email/SMS and chatting with tutor and students, taking video classes, and AI-based website navigation and FAQ answering.

2.2 Product High-Level Summary

Authentication and access control: Users register and log in with OTP verification. Each role (student, teacher, admin) gets its own dashboard and permissions.

Tutor verification: Teachers upload documents, and their profile is marked pending or verified by the admin.

Free trial request: Students can request for demo class to understand whether the tutor fits as per their requirement.

Communication and live classes: Matched users can chat, make audio calls, and teach in a video room with screen sharing. The video room will also have an in-built chat box.

Payments and earnings: Students pay online, and teachers view monthly earnings after the platform commission. There will be a payment gateway implemented by using third-party API.

AI chatbot: A chatbot helps in answering FAQ's and helps users to know about the website.

Scheduling and attendance: The system manages recurring classes, prevents double booking, tracks attendance, and enforces cancellation rules with a reliability score.

Feedback and reports: Students rate teachers, admins monitor feedback, and parents receive progress reports twice a month.

Reminders and notifications: Users get in-app and email alerts for sessions, payments, and approvals.

Support system: If the user faces a problem in any action or activity in the website then they can report to the admin to fix the issue.

2.3 Users and Characteristics

Student: Posts tuition requests, searches for tutors, pays, attends classes, and gives ratings about tutors.

Tutor: Gets verified, posts offers, accepts requests, teaches, and tracks earnings.

Parent: Receives periodic progress reports by email for school and college students. Not necessary for students above college level.

Admin: Manages users, approves verification, and monitors feedback and platform quality.

2.4 Operating Environment

Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile, with an internet connection. Camera and microphone access are needed for video and audio calls. And as for the server, we will be using Python 3.10 or later running FastAPI (served with Uvicorn), with a MongoDB database, on any OS that supports Python. For demo, we will be hosting the application in vercel or render.

2.5 Design and Implementation Constraints

This project follows the constraints defined in the CSE470 Project Outline, including mandatory MVC architecture and the approved tech-stack policy.

(a) Tech stack: FastAPI (Backend), React (Frontend), MongoDB (Database).

(b) MVC structure:

- **Model:** Data models and database access code (models/, schemas/, repositories/) that represent and store data such as users, requests, sessions, and payments.
- **View:** The React frontend (frontend) with pages and components.
- **Controller:** FastAPI route handlers (routers/ or controllers/) that receive requests from React and return responses, with business logic kept in a services/ layer.

(c) AI-backed feature: The AI tutor-finder chatbot is part of the FastAPI backend and calls an external AI API (Gemini or Groq) to suggest suitable teachers.

(d) Faculty-approved exception: None. If faculty disapproves of FastAPI as a backend language, then we need to think of other alternatives.

2.6 Assumptions and Dependencies (Optional)

Backup apis are considered if an api feature fails for some reason

3. System Architecture

3.1 Architectural Overview

[Insert or describe an MVC architecture diagram showing how the Model, View, and Controller layers are organized in your project, and how they communicate.]

3.2 Technology Stack

Frontend	React
Backend	FastAPI (Python)
Database	MongoDB(no sql)
Deployment	Local servers(such as- render or vercel) for the demo
External services	Payment gateway(SSLZ Commerz), NagerAPI(calender), email/SMS, video SDK, AI API (Gemini / Groq)

3.3 Class Diagram

[Draw the Class Diagram of your project showing all the classes and relationships necessary to implement your system]

4. Requirements and Features

List every requirement and its associated feature(s) below. Per the Project Outline (Section 3.2.1), all CRUD operations on the same entity by the same actor using the same logic count as ONE feature — do not split them. A minimum of 20 features is required.

[Replace the sample row with your own rows. Add as many rows as needed. Each Feature ID should be unique (F1, F2, F3…) and referenced consistently across this document and your Git commit history.]

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R1	User login, signup and session management	F1: Users signup and login and a session management authentication keeps the user logged in for a definite period of time and then it auto signs the user out.	Student, Teacher, Parents	Model: User; View: RegisterPage, LoginPage; Controller: AuthController	Pending	Planned

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R2	Users can verify their identity with a one-time code OTPs which expires after a short time	F2:Generate a random OTP and send it by email/phone at registration and login Automatically invalidate the OTP after 5 minutes	Student, Teacher, Parents	Model: OTP; View: OtpPage; Controller: AuthController	Pending	Planned
R3	Each role has its own access and dashboard	F3: Separate dashboards and role-restricted access for students, teachers, and admins	Student, Teacher, Admin, parents	Model: User, Role; View: StudentDashboard, TeacherDashboard, AdminDashboard; Controller: AuthController, DashboardController	Pending	Planned
R4	Teachers must be verified	F4: Teacher uploads certificates and ID; the admin reviews them and approves or rejects, and the status becomes pending, verified, or rejected	Teacher	Model: TutorProfile, VerificationDocument; View: VerificationPage, AdminVerificationPage; Controller: VerificationController, AdminController	Pending	Planned

R5	Users can post tuition requests	F5: Create, view, update, and delete tuition requests or ads	Student, Teacher	Model: TuitionRequest; View: PostRequestPage, RequestListPage; Controller: RequestController	Pending	Planned

Req. ID	**Requirement**	**Feature Description**	**Actor**	**MVC Components Touched**	**Sprint Completed**	**Status**
R6	Users can filter by budget and other factors	F6: Filter listings by the teacher's fee or the student's budget, subject	Student, Teacher	Model: TuitionRequest; View: RequestListPage; Controller: RequestController	Pending	Planned
R7	Users can report problems to the admin	F7: Support system where a user submits a report about any problem (such as website, payment, class, or tutor issues); the admin views it, replies, and updates its status (open, in progress, resolved)	Student, Teacher, Admin, Parents	Model: SupportTicket, Notification; View: SupportPage, AdminSupportPage; Controller: SupportController, AdminController	Pending	Planned

R8	Free Trial Class Request	F8: A student can request one free demo session with a teacher by proposing a date and time. The teacher accepts or declines, and the request status (pending, accepted, declined) is updated with a notification to both sides. Only one trial is allowed per student-teacher pair. An accepted trial takes place in the video-conference room.	Student, Teacher	Model: TrialRequest (student_id, teacher_id, proposed_time, status), Notification. View: React trial request form, teacher's trial inbox with accept/decline buttons. Controller: trial router (create, accept, decline, list requests).	Pending	Planned
R9	Teachers can hold online classes	F9: Video-conference room with screen sharing and in-room chat	Teacher, Student	Model: ClassRoom; View: VideoRoomPage; Controller: RoomController	Pending	Planned

Req. ID	**Requirement**	**Feature Description**	**Actor**	**MVC Components Touched**	**Sprint Completed**	**Status**
R10	Students can rate teachers	F10: Submit a 1-5 rating and a review; the teacher's average rating updates	Student	Model: Review, TutorProfile; View: ReviewForm, TutorProfilePage; Controller: ReviewController	Pending	Planned
R11	Students can pay teachers online	F11: Pay through SSLCommerz or Stripe; the order is marked paid after confirmation	Student	Model: Order, Transaction; View: CheckoutPage; Controller: PaymentController	Pending	Planned

R12	AI Chatbot (FAQ & Website Guide)	F12: A chatbot answers frequently asked questions and helps users learn how the website works (registration, OTP login, posting a tuition request, payments, free trial, etc.). Users type a question and get an instant reply. If the bot can't answer, it suggests contacting support.	Student, Teacher, Parent	Model: ChatMessage (user_id, question, reply, timestamp), FAQ (question, answer). View: floating React chat widget with message history and input box. Controller: chatbot router (receive message, add FAQ context to the prompt, call the AI API, return reply).	Pending	Planned
R13	Teachers can track their earnings	F13: Monthly income per student after the configurable platform commission	Teacher	Model: Transaction, PlatformConfig ; View: EarningsDashboard; Controller: EarningsController	Pending	Planned

Req. ID	**Requirement**	**Feature Description**	**Actor**	**MVC Components Touched**	**Sprint Completed**	**Status**
R14	Admins can monitor feedback quality	F14: Summary view of all feedback with average scores and flagged negative reviews	Admin	Model: Review; View: AdminFeedbackPage; Controller: AdminController	Pending	Planned

R15	Users can see past payments	F15: Paged payment history list	Student, Teacher	Model: Transaction; View: PaymentHistoryPage; Controller: PaymentController	Pending	Planned
R16	Cancellations follow notice rules	F16: Cancel or reschedule within the notice period; late cancellations lower the reliability score	Student, Teacher	Model: Session, User; View: SessionPage; Controller: SessionController	Pending	Planned
R17	Parents receive progress reports	F17: Automatic report twice a month (attendance, homework, scores, session summary) updated in app or sent to parent's e-mail	Parent	Model: Session, Attendance, Report; View: ReportPage (PDF/email); Controller: ReportController	Pending	Planned
R18	Sessions are scheduled without conflicts	F18: Recurring weekly calendar, double-booking detection, and Google Calendar sync	Teacher, Student	Model: Session, Calendar; View: CalendarPage; Controller: CalendarController	Pending	Planned

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R19	Users receive reminders	F19: In-app and email reminders for sessions, payments, and approvals	Student, Teacher	Model: Notification; View: NotificationCenter; Controller: NotificationController	Pending	Planned
R20	Attendance confirms class completion	F20: Check-in and check-out by both teacher and student; session is marked completed when both confirm	Student, Teacher	Model: Session; View: SessionPage; Controller: AttendanceController	Pending	Planned

4.1 Feature Count Summary

Minimum features required (per Project Outline, Section 7)	20
Total features listed in Section 4 above	20
Total features fully completed (“Done” per Section 7.2 of the Project Outline)	0 (work in progress)

5. External Interface Requirements (Optional)

Interface Type	Description
User Interface	Pages should load in a few seconds. Slow work(AI search, emails) run in the background
Hardware Interface	No special hardware is needed. Users need a device with a web browser, and a camera and microphone for video and audio calls.

Software Interface (APIs, third-party services)	Payment gateway (SSLCommerz / Stripe) for online payments. Google Calendar for schedule syncing. Email/SMS service for OTP codes, reminders, and parent reports. Video SDK (e.g., Jitsi) for live classes. AI API (Gemini / Groq) for website navigation and FAQs’

Interface Type	**Description**
Communication Interface	The React frontend and FastAPI backend communicate over HTTPS using REST APIs with JSON data. Real-time chat uses WebSockets. Audio and video calls use WebRTC through the video SDK. Email is sent over SMTP.

6. Non-Functional Requirements (Optional)

Category	**Requirement**
Performance	Pages should load within about 3 seconds under normal use. Slow tasks such as AI suggestions and email sending run in the background so they do not block the user
Security	Users log in with OTP verification and token based sessions. Passwords are stored hashed. Each role can access only its own features. All data travels over HTTPS. Uploaded documents are stored securely, and payments are handled only by the payment gateway, so card details are never stored.
Usability	The interface is simple and works on both desktop and mobile browsers. Users can complete main tasks, such as posting a request or booking a class, in a few steps, with clear error messages.
Reliability / Availability	The system should be available during normal use, with only brief downtime for maintenance. Payments, bookings, and attendance records must stay consistent even if a request fails, and database backups protect against data loss.
Scalability	The system can handle more users by adding server capacity. The separate frontend, backend, and database let each part scale on its own, and MongoDB can grow to hold more data.

7. Sprint History and Review Ceremony Log

Record the goals and attainment shared during each weekly sprint review ceremony, per Section 7.1 of the Project Outline.

Sprint	Goals Set (Start of Week)	Attainment Reported (End of Week)	Features Completed This Sprint
Sprint 1			
Sprint 2			
Sprint 3			
Sprint 4			

7.1 Deviations and Lessons Learned

[Briefly explain any sprint where attainment did not match the planned goals, why it happened, and what was adjusted for the following sprint.]

8. Testing Summary

[Briefly describe how each major feature was tested (manual testing steps, sample test cases, or automated tests). A short table or list is acceptable.]

9. AI Tool Usage Disclosure

Per the Project Outline's Academic Integrity & AI Usage Policy (Section 6), disclose every instance where a generative AI tool was used during development. Leave the table empty (do not delete it) if no AI tool was used.

Feature / Task	AI Tool Used	How It Was Used (be specific)	I Can Fully Explain This Code (Yes/No)

[By submitting this document, you confirm that you are able to personally explain, modify, and debug every piece of code listed above during your viva.]

10. Individual Contribution Summary

[Since each student develops their project individually, summarize your own work: which features you implemented, approximate time spent per sprint, and any challenges you solved independently. If you collaborated with your 4-student group on non-code matters (e.g. idea discussion), state that clearly and distinguish it from your individual implementation work.]

11. Pre-Submission Checklist

Checklist Item	Confirmed (Yes/No)
The application implements the MVC architecture	
At least 20 features are listed and each is correctly counted per the granularity rule	
No forbidden framework (e.g. Django, Flask as primary backend) was used	
All four sprints' goals and attainment are recorded in Section 7	
The GitHub repository is up to date and the section faculty is added as a collaborator	
The AI Usage Disclosure table (Section 10) is complete and honest	

Checklist Item	**Confirmed (Yes/No)**
I can personally explain and defend every feature listed in this document during the viva	